

ASSIGNMENT- 10.5

Name:MD Arshad

HT.No: 2303A51334

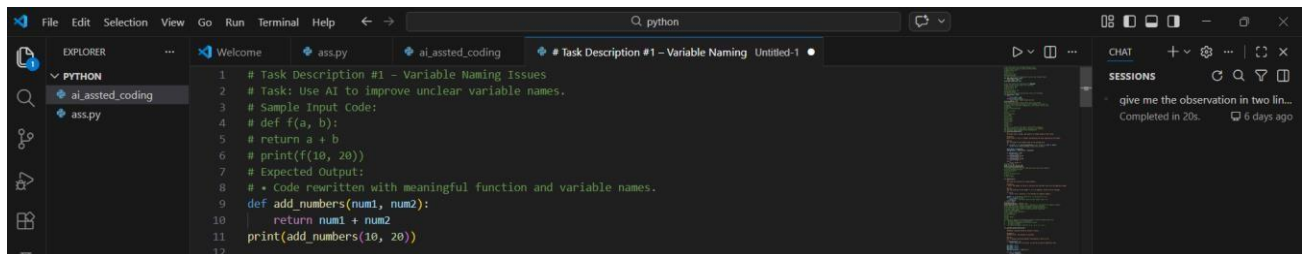
Batch: 20

Task Description #1 – Variable Naming Issues

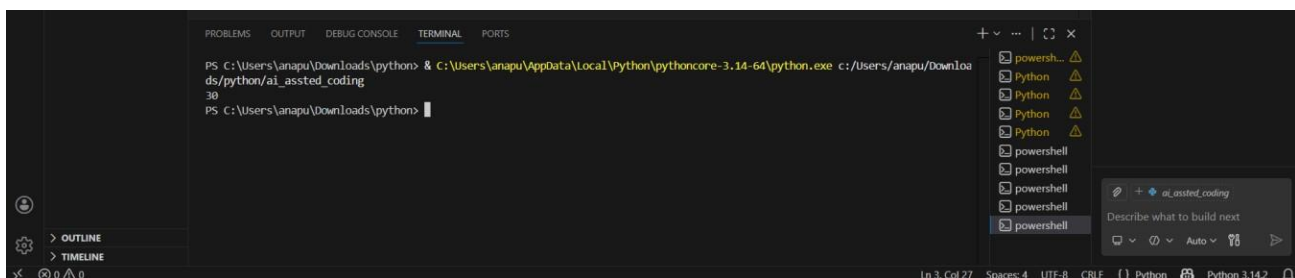
Sample input :

```
def f(a,b):  
    return a+b  
Print(f(10,20))
```

Code & probmt



Result:



Observation:

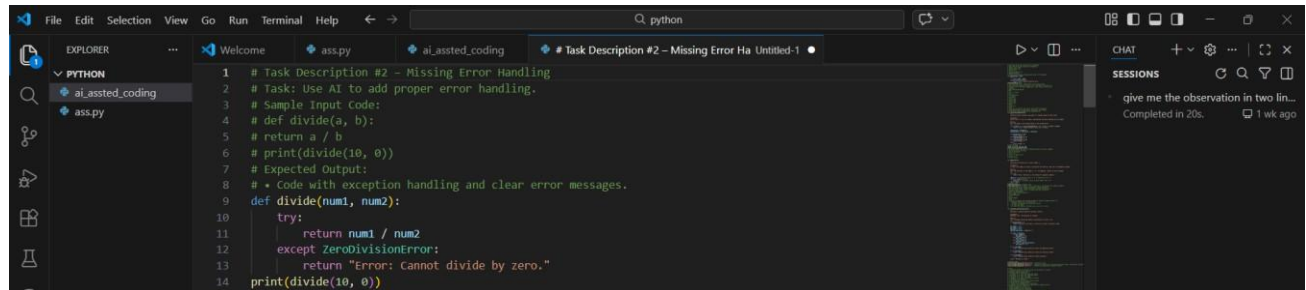
The original code used unclear and non-descriptive names, making it hard to understand the function's purpose. Using meaningful names improves clarity, readability, and overall code quality.

Task Description #2 – Missing Error Handling

Sample Input Code:

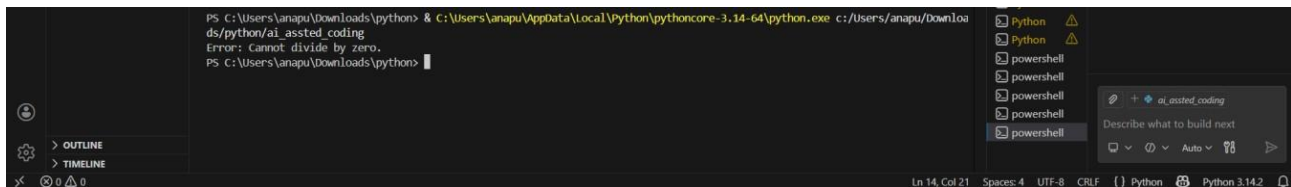
```
# def divide(a, b):  
#     return a / b  
# print(divide(10, 0))
```

Prompt & Code:



```
1 # Task Description #2 - Missing Error Handling
2 # Task: Use AI to add proper error handling.
3 # Sample Input code:
4 # def divide(a, b):
5 #     return a / b
6 # print(divide(10, 0))
7 # Expected Output:
8 # * Code with exception handling and clear error messages.
9 def divide(num1, num2):
10     try:
11         return num1 / num2
12     except ZeroDivisionError:
13         return "Error: Cannot divide by zero."
14 print(divide(10, 0))
```

Result:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore\3.14-64\python.exe c:\Users\anapu\Downloa
ds\python\ai_assisted_coding
Error: Cannot divide by zero.
PS C:\Users\anapu\Downloads\python> |
```

Observation:

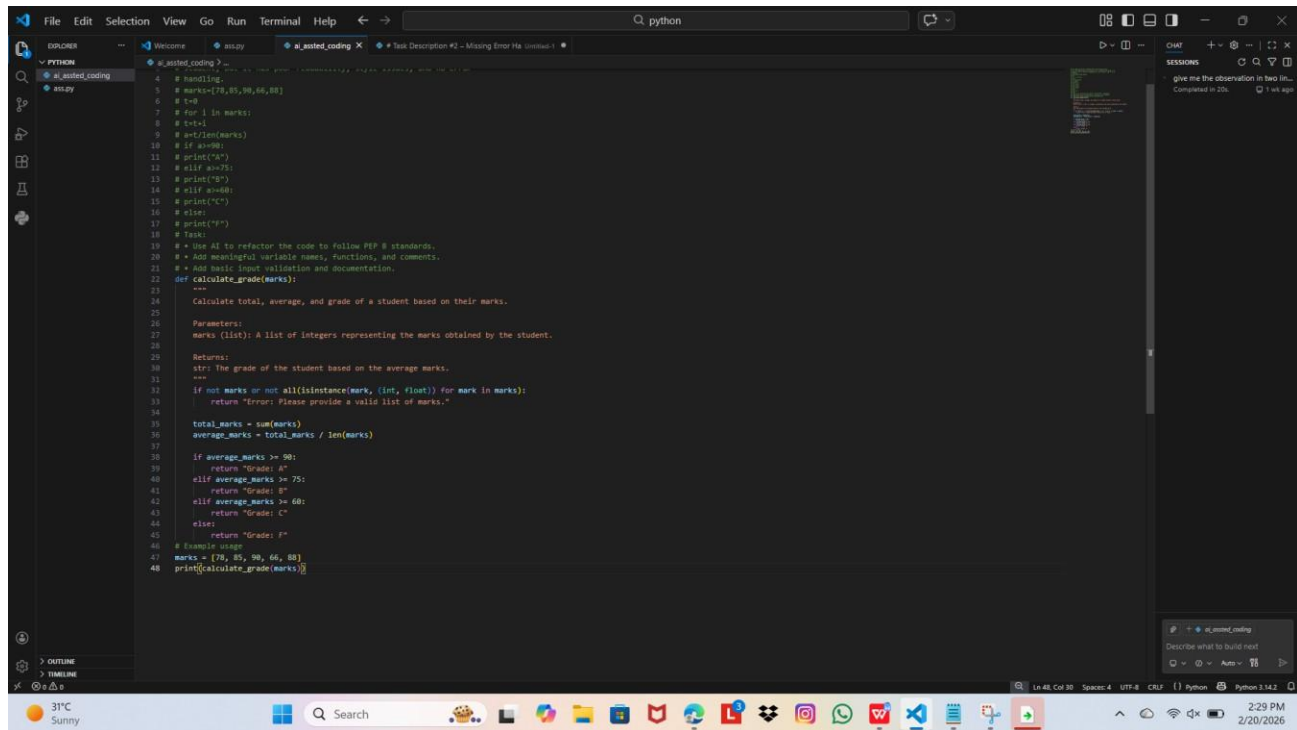
The original code did not handle division by zero, which could cause the program to crash. Adding exception handling makes the program safer and provides a clear, user-friendly error message instead of stopping abruptly.

Task Description #3: Student Marks Processing System

Sample input code:

```
marks=[78,85,90,66,88]
t=0
for i in marks:
    t=t+i
a=t/len(marks)
if a>=90:
    print("A")
elif a>=75:
    print("B")
elif a>=60:
    print("C")
else:
    print("F")
```

Prompt & Code:



The screenshot shows a code editor with two tabs: 'ai_assisted_coding' and 'ai_assisted_coding - Refactored'. The 'ai_assisted_coding' tab contains the original code, which is a simple script to calculate the average of a list of marks and print the grade. The 'ai_assisted_coding - Refactored' tab contains the refactored code, which uses meaningful variable names, proper formatting, functions, and clear documentation following PEP 8 standards. The refactored code also includes basic input validation to ensure the marks are integers or floats.

```
# Original Code (ai_assisted_coding):
# Handling
# marks=[78,85,90,66,88]
# t=0
# for i in marks:
#     t+=i
# total=t
# avg=t/len(marks)
# if avg>90:
#     print("A")
# elif avg>75:
#     print("B")
# elif avg>60:
#     print("C")
# else:
#     print("F")
# print(avg)

# Refactored Code (ai_assisted_coding - Refactored):
# Use AI to refactor the code to follow PEP 8 standards.
# Add meaningful variable names, functions, and comments.
# Add basic input validation and documentation.
def calculate_grade(marks):
    """
    Calculate total, average, and grade of a student based on their marks.

    Parameters:
    marks (list): A list of integers representing the marks obtained by the student.

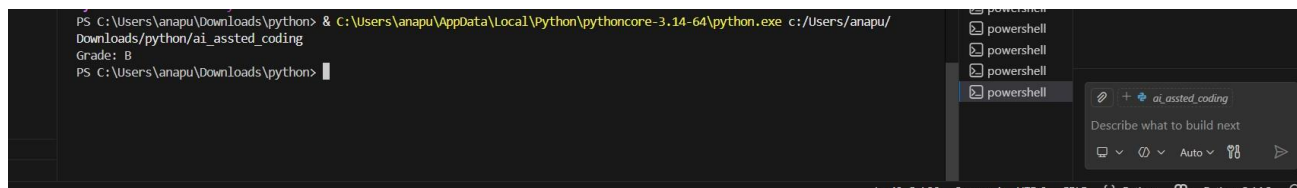
    Returns:
    str: The grade of the student based on the average marks.
    """
    if not marks or not all(isinstance(mark, (int, float)) for mark in marks):
        return "Error: Please provide a valid list of marks."

    total_marks = sum(marks)
    average_marks = total_marks / len(marks)

    if average_marks >= 90:
        return "Grade: A"
    elif average_marks >= 75:
        return "Grade: B"
    elif average_marks >= 60:
        return "Grade: C"
    else:
        return "Grade: F"

# Example usage
marks = [78, 85, 90, 66, 88]
print(calculate_grade(marks))
```

Result:



The screenshot shows a PowerShell terminal window with the following commands and output:

```
PS C:\Users\anapu\Downloads> python c:\Users\anapu\Downloads\python\ai_assisted_coding
Grade: B
PS C:\Users\anapu\Downloads\python>
```

Observation:

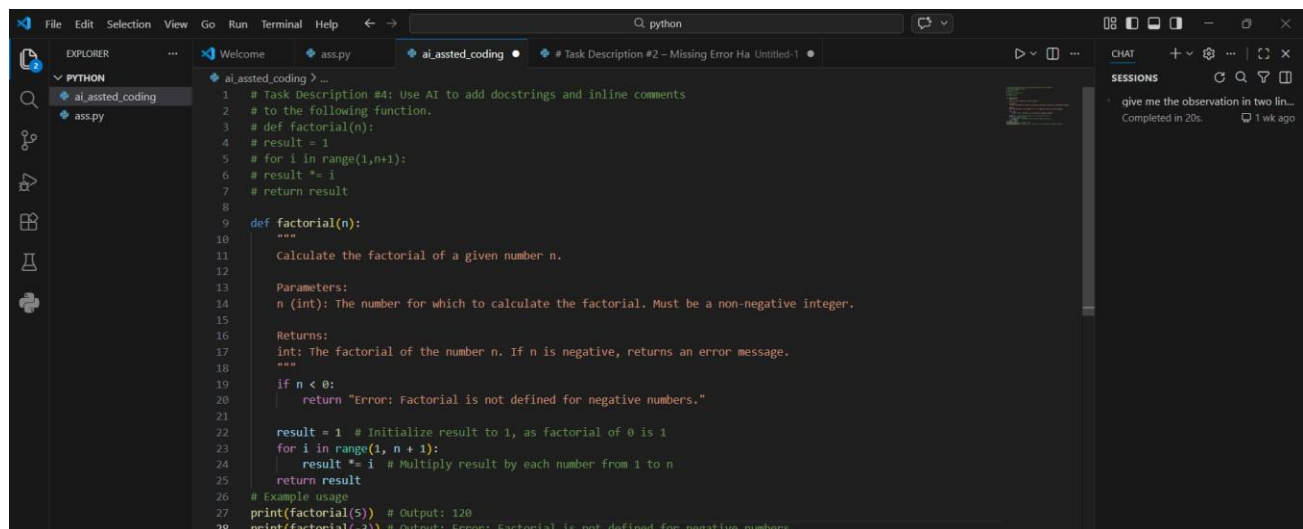
The refactored code improves readability by using meaningful variable names, proper formatting, functions, and clear documentation following PEP 8 standards. It also adds basic input validation, making the program more reliable and user-friendly by preventing errors from invalid data.

Task Description #4: Use AI to add docstrings and inline comments

Sample input:

```
def factorial(n):  
  
    result = 1  
  
    for i in range(1,n+1):  
  
        result *= i  
  
    return result
```

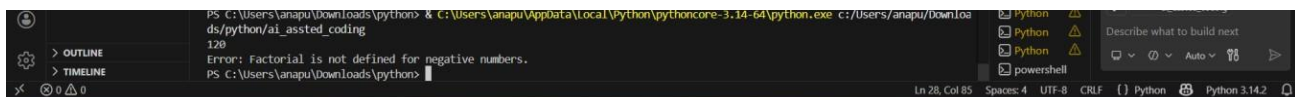
Prompt & Code:



The screenshot shows a VS Code editor with a Python file named `ai_assted_coding.py`. The code is as follows:

```
1 # Task Description #4: Use AI to add docstrings and inline comments  
2 # to the following function.  
3 # def factorial(n):  
4 # result = 1  
5 # for i in range(1,n+1):  
6 # result *= i  
7 # return result  
8  
9 def factorial(n):  
10     """  
11     Calculate the factorial of a given number n.  
12  
13     Parameters:  
14     n (int): The number for which to calculate the factorial. Must be a non-negative integer.  
15  
16     Returns:  
17     int: The factorial of the number n. If n is negative, returns an error message.  
18     """  
19     if n < 0:  
20         return "Error: Factorial is not defined for negative numbers."  
21  
22     result = 1 # Initialize result to 1, as factorial of 0 is 1  
23     for i in range(1, n + 1):  
24         result *= i # Multiply result by each number from 1 to n  
25     return result  
26  
27 # Example usage  
28 print(factorial(5)) # Output: 120  
29 print(factorial(-3)) # Output: Error: factorial is not defined for negative numbers.
```

Result:



The screenshot shows a terminal window with the following output:

```
PS C:\Users\anapu\Downloads\python> & c:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\anapu\Downloads\python\ai_assted_coding  
120  
Error: Factorial is not defined for negative numbers.  
PS C:\Users\anapu\Downloads\python>
```

Observation

The updated function includes a clear docstring and inline comments, which improve understanding of the code's purpose and logic.

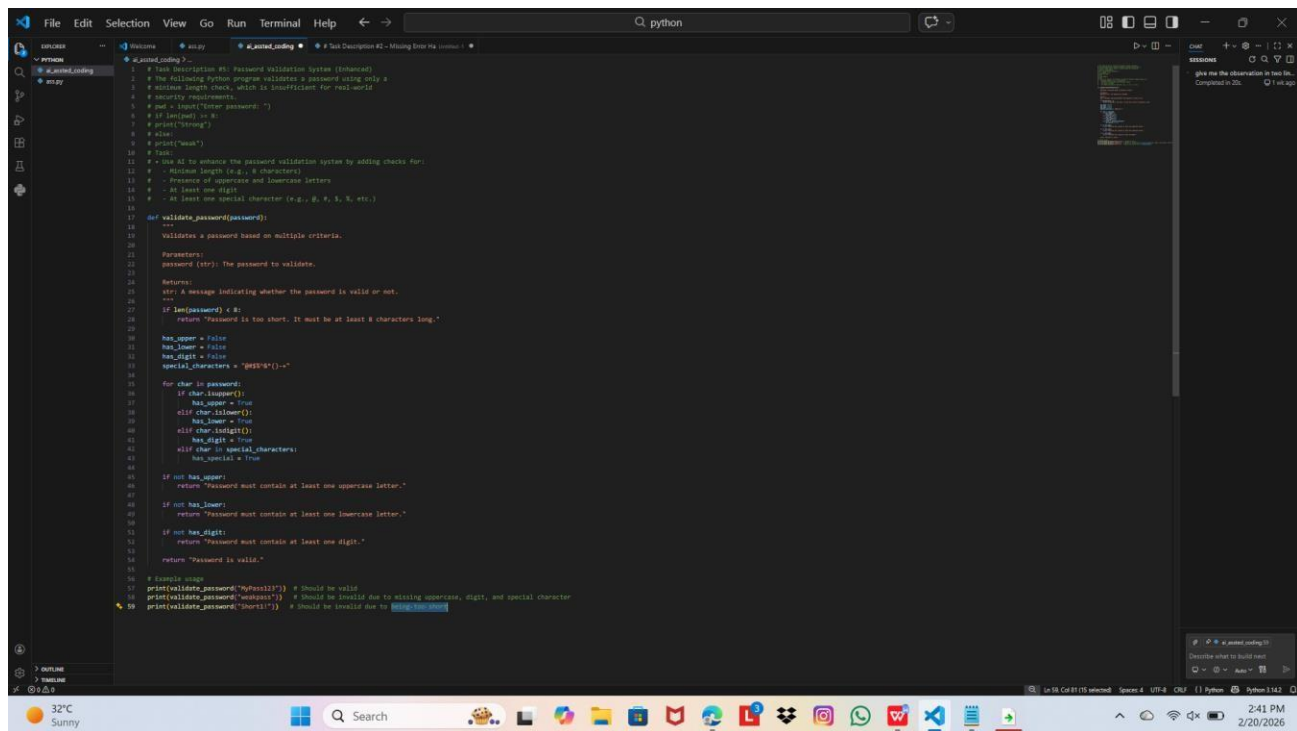
It also adds basic validation for negative numbers, making the function more informative and reliable.

Task Description #5: Password Validation System (Enhanced)

Sample input:

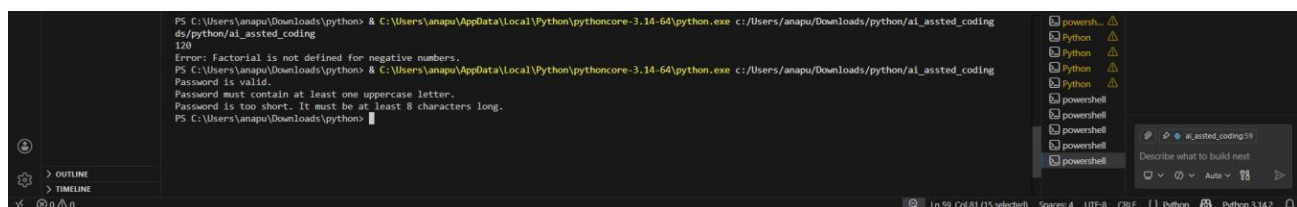
```
pwd = input("Enter password: ")  
  
if len(pwd) >= 8: print("Strong")  
  
else:  
  
print("Weak")
```

Prompt & Code:



```
1 # Task Description #5: Password Validation System (Enhanced)
2 # The following Python program validates a password using only a
3 # minimum length check, which is insufficient for real-world
4 # security requirements.
5 # pwd = input("Enter password: ")
6 # if len(pwd) >= 8:
7 #     print("Strong")
8 # else:
9 #     print("Weak")
10 # Task:
11 # Use AI to enhance the password validation system by adding checks for:
12 # - Minimum length (e.g., 8 characters)
13 # - Presence of uppercase and lowercase letters
14 # - At least one digit
15 # - At least one special character (e.g., !, @, #, $, %, etc.)
16
17 def validate_password(password):
18     """
19     Validates a password based on multiple criteria.
20
21     Parameters:
22     password (str): The password to validate.
23
24     Returns:
25     str: A message indicating whether the password is valid or not.
26     """
27     if len(password) < 8:
28         return "Password is too short. It must be at least 8 characters long."
29
30     has_upper = False
31     has_lower = False
32     has_digit = False
33     special_characters = "!@#$%^&*()-+~"
34
35     for char in password:
36         if char.isupper():
37             has_upper = True
38         elif char.islower():
39             has_lower = True
40         elif char.isdigit():
41             has_digit = True
42         elif char in special_characters:
43             has_special = True
44
45     if not has_upper:
46         return "Password must contain at least one uppercase letter."
47     if not has_lower:
48         return "Password must contain at least one lowercase letter."
49     if not has_digit:
50         return "Password must contain at least one digit."
51     if not has_special:
52         return "Password must contain at least one special character."
53     return "Password is valid."
54
55 # Example usage
56 print(validate_password("MyPass123!")) # Should be valid
57 print(validate_password("mypass"))    # Should be invalid due to missing uppercase, digit, and special character
58 print(validate_password("12345678"))  # Should be invalid due to missing uppercase, digit, and special character
```

Result:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assisted_coding
ds/python/ai_assisted_coding
120
Error: Factorial is not defined for negative numbers.
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assisted_coding
Password is valid.
Password must contain at least one uppercase letter.
Password is too short. It must be at least 8 characters long.
PS C:\Users\anapu\Downloads\python>
```

Observation:

The enhanced password validation improves security by checking for length, uppercase and lowercase letters, digits, and special characters instead of just minimum length.

However, there is a small issue: the variable `has_special` is used without being initialized and the final check for a special character is missing, which could cause an error.